

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSESSMENT TOOL 2: CASE STUDY

C03: Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications.Total Marks: 30 | Question Count: 3

Student Name	T . Kodanda Ram
Register Number	192524162

Case Study 1: Smart Retail Inventory Management System

The retail chain requires a centralized cloud-based inventory management system connecting multiple stores and their devices. POS systems and dashboards communicate with cloud services through APIs. The system stores product information, transaction logs, and analytics data and supports demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways.

1. Understanding of the Case

Main technical and business requirements:

- Connect POS systems and dashboards to centralized cloud services.
- Store product information, transaction logs, and analytics datasets securely.
- Process inventory and sales data from multiple stores.
- Support analytics and demand forecasting.
- Provide APIs for communication between clients and cloud services.
- Support secure access using VPNs, firewalls, and API gateways.
- Provide scalable and reliable operation.

2. Application of Cloud Concepts

Requirement	Cloud Solution
POS and dashboard access	Web/mobile clients
Communication	RESTful/Web APIs
Product and transaction storage	Cloud database and object storage
Analytics	Cloud analytics platform

Demand forecasting	Cloud-based analytics/ML service
API security	API Gateway
Network security	VPN and Firewall
Scalability	Auto-scaling cloud resources
Monitoring	Cloud monitoring service

POS systems send transactions through an API gateway to backend cloud services. The data is stored in cloud databases/storage and processed by analytics services for inventory analysis and demand forecasting.

3. Analysis and Trade-offs

Advantages:

Scalability: Cloud resources can increase when transaction volumes become high.

Centralized data: All stores can access updated inventory information.

Better forecasting: Historical sales data can support future demand prediction.

Reduced infrastructure management: Physical server maintenance is reduced.

Improved accessibility: Managers can access dashboards from different locations.

Trade-offs:

Security vs accessibility: Remote access requires stronger authentication and network security.

Cost vs scalability: Auto-scaling improves performance but can increase cloud costs.

Real-time processing vs complexity: Real-time updates require more sophisticated services.

Cloud dependency: Reliable network connectivity is important.

4. Proposed Solution Design

Case Study 1: Proposed Cloud Architecture

POS / Store Devices	API Gateway	Authentication + Firewall	Cloud Application Services	Database / Storage	Analytics + Forecasting	Management Dashboard
↓	↓	↓	↓	↓	↓	

--	--	--	--	--	--	--

POS Systems / Store Devices → API Gateway → Authentication + Firewall → Cloud Application Services → Cloud Database / Storage → Cloud Analytics → Demand Forecasting → Management Dashboard

Working:

POS systems generate sales and inventory transactions.

Transactions are sent securely through the API Gateway.

Authentication verifies authorized clients.

Application services process requests.

Product and transaction information is stored in cloud storage/database.

Analytics services process historical and real-time data.

Demand forecasting predicts future product requirements.

Dashboards display inventory levels and forecasting results.

Justification: The design combines cloud storage, Web APIs, analytics/platform services, and security mechanisms, making it practical for centralized retail inventory management.

Case Study 2: Cloud-Based Document Collaboration System

The enterprise requires a system similar to Google Docs where users can create, edit, share, and access documents through browsers. Documents are stored in distributed cloud storage with version control. Web APIs support synchronization across multiple clients and devices. Low latency, high availability, encryption, identity management, and access permissions are required.

1. Understanding of the Case

Provide browser-based document access.

Support real-time collaborative editing.

Allow document sharing among authorized users.

Store documents using distributed cloud storage.

Maintain document versions.

Synchronize changes across devices.

Provide low-latency and highly available access.

Protect data using encryption and access controls.

2. Application of Cloud Concepts

Requirement	Cloud Solution
User interface	Web application

Communication	Web APIs
Document storage	Distributed cloud storage
Real-time editing	Application/backend services
Synchronization	Web APIs / real-time synchronization
Version control	Document versioning
Authentication	Identity and Access Management
Security	Encryption
Global performance	Distributed infrastructure/CDN
Availability	Replication and load balancing

3. Analysis and Trade-offs

Advantages:

Real-time collaboration: Multiple users can work on documents simultaneously.

Accessibility: Documents can be accessed from different devices.

Version recovery: Previous versions can be restored.

Scalability: Cloud resources can support increasing users and documents.

High availability: Distributed infrastructure reduces dependence on a single server.

Trade-offs:

Consistency vs performance: Immediate synchronization requires additional processing and network communication.

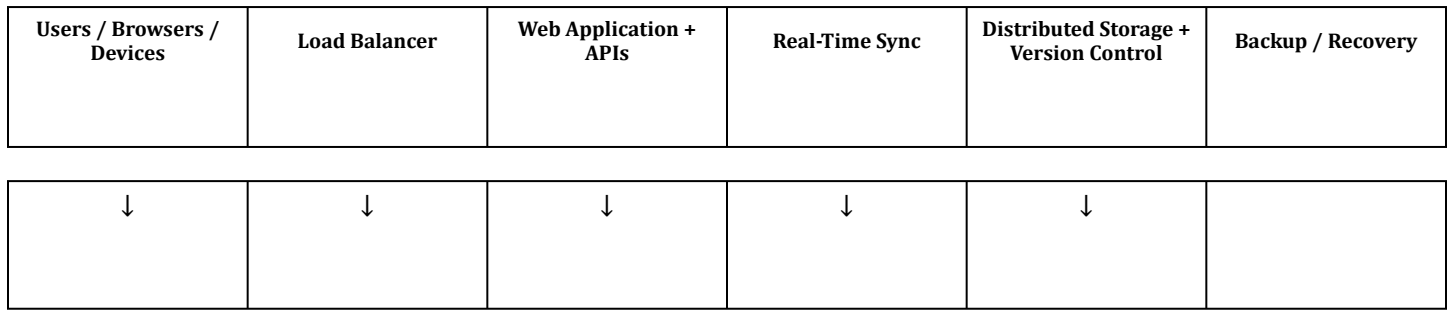
Availability vs cost: Replication and redundant infrastructure increase resource requirements.

Security vs convenience: Strong authentication and permissions add complexity.

Global performance vs infrastructure cost: Distributed services improve latency but require more resources.

4. Proposed Solution Design

Case Study 2: Proposed Cloud Architecture



Users/Browsers/Mobile Devices → Load Balancer → Web Application → Web API Services → Real-Time Sync Service → Distributed Storage + Version Control → Backup/Recovery

Security flow: User → Identity Authentication → Authorization/Permissions → Encrypted HTTPS → Cloud Application

Working:

The user logs into the collaboration platform.

Identity management verifies the user's credentials.

The user opens a document through a browser.

The application retrieves the document from distributed cloud storage.

Edits are sent through Web APIs/real-time synchronization services.

Other authorized users receive updated content.

Previous versions are maintained through version control.

Encryption protects documents during transmission and storage.

Justification: The architecture satisfies the requirements for real-time editing, distributed storage, version control, API synchronization, global availability, encryption, identity management, and access control.

Case Study 3: Online Food Ordering Platform

The startup needs a cloud-based food ordering system accessible through web browsers and mobile clients. Frontend clients communicate with backend services using RESTful Web APIs. Cloud storage maintains menu images, invoices, and customer data. Virtual machines and managed Kubernetes provide scalability and reliability. Authentication tokens, HTTPS, and role-based access control provide security.

1. Understanding of the Case

Support web and mobile clients.

Provide RESTful Web APIs.

Store customer, order, invoice, and menu information securely.

Provide scalable backend infrastructure.

Support high availability and reliability.

Use authentication tokens for secure access.

Protect communication using HTTPS.

Use role-based access control.

2. Application of Cloud Concepts

Requirement	Cloud Solution
Customer application	Web/mobile frontend
Backend communication	RESTful APIs
Application hosting	Virtual machines
Container orchestration	Managed Kubernetes
Menu images	Cloud object storage
Customer/order data	Cloud database
Authentication	Token-based authentication
Communication security	HTTPS
Authorization	Role-Based Access Control (RBAC)
Scalability	Kubernetes auto-scaling
Reliability	Multiple instances/replicas

3. Analysis and Trade-offs

Advantages:

Scalability: Kubernetes can increase or decrease application instances based on demand.

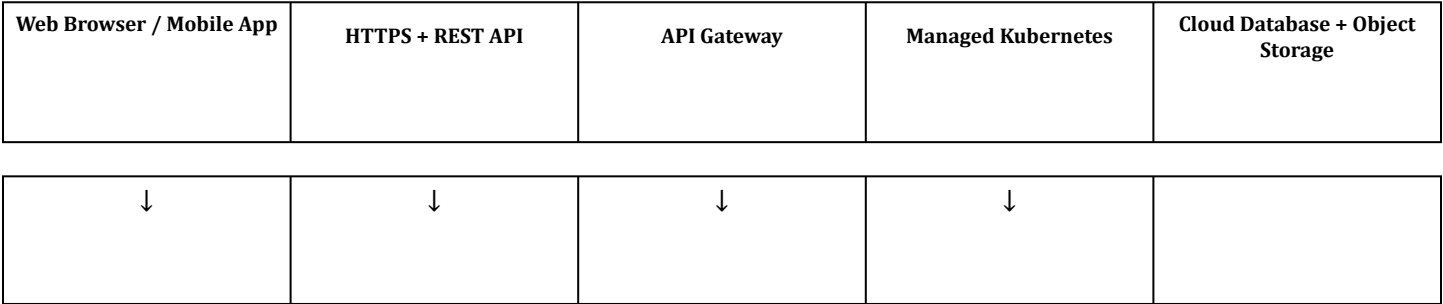
Reliability: Multiple application instances can continue serving users if one fails.

Accessibility: Customers can order using web or mobile applications.

- Secure storage: Customer and order information can be stored in controlled cloud services.
- API-based architecture: Frontend and backend can be developed and scaled independently.
- Trade-offs:
 - Scalability vs cost: Additional VM/container resources during peak periods increase cost.
 - Kubernetes flexibility vs complexity: Kubernetes provides scalability but requires more management knowledge.
 - Security vs usability: Authentication and authorization improve security but add complexity.
 - High availability vs resources: Multiple instances require additional computing resources.

4. Proposed Solution Design

Case Study 3: Proposed Cloud Architecture



Web Browser + Mobile App → HTTPS → RESTful API → API Gateway → Managed Kubernetes → Order/Payment/User Services → Cloud Database + Cloud Object Storage

Security flow: Client → HTTPS → Authentication Token → RBAC → REST API → Backend Services

Working:

- The customer opens the web or mobile application.
 - The application communicates with backend services using RESTful APIs.
 - HTTPS encrypts communication.
 - Authentication tokens verify the customer's identity.
 - RBAC determines which resources the user can access.
 - The backend processes menu, order, customer, and invoice operations.
 - Menu images and invoices are stored in cloud storage.
 - Customer and order information is stored in a cloud database.
 - Managed Kubernetes distributes workloads and provides scalability and reliability.
- Justification: The architecture directly uses RESTful Web APIs, cloud storage, virtual machines, managed Kubernetes, authentication tokens, HTTPS, and RBAC, making it suitable for a scalable online food-ordering platform.

Overall Comparison

Feature	Smart Retail	Document Collaboration	Food Ordering
Main purpose	Inventory management	Real-time collaboration	Online ordering
Main API	Web APIs	Web APIs	RESTful APIs
Storage	Product & transaction data	Documents & versions	Images, invoices & customer data
Major platform service	Analytics & forecasting	Real-time synchronization	Managed Kubernetes
Key security	VPN, firewall, API gateway	Encryption, IAM, permissions	Tokens, HTTPS, RBAC
Main challenge	Demand prediction	Real-time consistency	Peak-time scalability

Conclusion

All three case studies demonstrate how cloud storage, Web APIs, infrastructure, and platform services can be integrated to solve real-world business problems. The retail system focuses on centralized inventory and forecasting, the document system focuses on real-time collaboration and synchronization, and the food-ordering system focuses on scalable API-based services. The proposed designs consider important trade-offs between security, performance, scalability, availability, complexity, and cost, while providing feasible solutions for the given requirements.

Assessment Basis

This document is prepared from the uploaded CSA15 Assessment Tool 2 case-study document and follows its case-study rubric: Understanding of Case (25%), Application of Concepts (25%), Analysis (20%), Solution Design (20%), and Clarity (10%).